



DAYANANDA SAGAR COLLEGE OF ARTS, SCIENCE AND COMMERCE

DEPARTMENT OF COMPUTER APPLICATION (MCA)

A PROJECT REPORT ON:

Application of Theory of Computation (TOC) in Games:

Finite states machine controls the behavior of the NPC character in a video game

SUBMITTED BY:

BHADANI DHARM P03CJ24S126017

GHASADIYA MAULIK P03CJ24S126035

UNDER THE GUIDANCE OF:

Mrs. AKSHATHA DEEPAK

Asst. Professor,

Department of Computer Applications

2024-25

Title: Application of Theory of Computation in Video Game Development

Finite State Machine (FSM) Based Behaviour Control for NPCs in Games

1. Introduction:

The **Theory of Computation (TOC)** provides formal foundations for designing systems that process input, manage state, and deliver outputs based on defined logic. One practical application of TOC in the field of **video game development** is the use of **Finite State Machines (FSMs)** to control the behaviour of **Non-Playable Characters (NPCs)**.

FSMs enable developers to simulate intelligent, reactive behaviour by representing an NPC's logic through discrete states and transitions. These transitions are governed by conditions such as the player's proximity, visibility, and health status, making FSMs an ideal solution for modelling simple AI in game.

2. Problem Statement

Designing believable NPC behavior is essential for creating immersive video game experiences. Without a structured method,

NPC behavior can become chaotic or too simplistic, reducing the game's appeal.

This project aims to create a simple yet intelligent NPC system using FSMs. The NPC should behave according to its current state and react appropriately to changing conditions in the game environment, such as:

- The distance between the NPC and the player.
- Whether the NPC can see the player (line-of-sight).
- The NPC's current health status.
- The player's actions and position.

3. Application of TOC in Game AI:

3.1 Finite State Machines (FSM)

FSMs model the logical flow of an NPC's behaviour. Each state represents a specific mode of behaviour, and transitions are triggered by game conditions:

- **State 0 (Idle):** NPC is stationary.
- **State 1 (Patrolling):** NPC moves along a predefined path.
- **State 2 (Chasing):** NPC moves towards the player when detected.
- **State 3 (Attacking):** NPC attacks when within range.
- **State 4 (Dead):** NPC stops all actions after health drops to zero.

3.2 Transition Triggers

- **From Idle to Patrolling:** Time elapsed or random interval.
- **From Patrolling to Chasing:** Player enters NPC's field of view.
- **From Chasing to Attacking:** Player is within attack range.
- **From Any State to Dead:** NPC's health reaches zero.

4. System Design Overview

4.1 Objectives

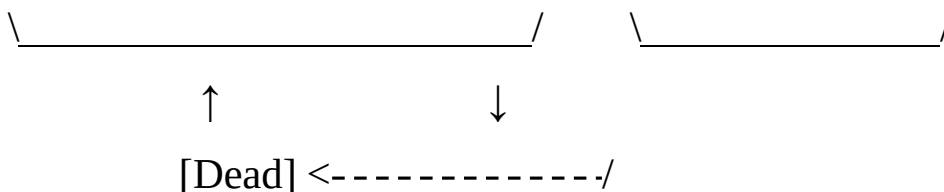
- Implement intelligent NPC behaviour using FSM.
- Define transitions based on player proximity and game events.
- Enhance game realism and interaction.

4.2 FSM Diagram (Conceptual)

markdown

CopyEdit

[Idle] ---> [Patrolling] ---> [Chasing] ---> [Attacking]



4.3 FSM Class Implementation in Pseudocode

python

CopyEdit

```
class NPCFSM:
```

```
    def __init__(self):
        self.state = "Idle"
```

```
def update(self, inputs):  
    if self.state == "Idle" and inputs["time_elapsed"]:  
        self.state = "Patrolling"  
    elif self.state == "Patrolling" and inputs["player_visible"]:  
        self.state = "Chasing"  
    elif self.state == "Chasing" and inputs["in_attack_range"]:  
        self.state = "Attacking"  
    elif inputs["health"] <= 0:  
        self.state = "Dead"
```

5. Expected Outcome

- The NPC dynamically responds to the player's actions.
- NPC transitions between defined states based on real-time game data.
- Demonstrates the use of FSMs for real-time behaviour modelling in games.

6. Tools and Technologies

- **Programming Language:** Python / C++ / JavaScript
(depending on engine)
- **Game Engine:** Unity / Unreal / Godot
- **FSM Modelling:** Custom logic or built-in engine FSM modules

7. Challenges and Limitations

- **Scalability:** FSMs become complex with more states.
- **Unpredictable Player Behavior:** May require dynamic or learning-based state adjustments.
- **Animation Sync:** Transitions must sync smoothly with character animations.

8. Future Enhancements

- **Hierarchical FSMs:** For better scalability and nested behaviours.
- **Behaviour Trees:** Extend FSMs for more complex decisions.
- **Machine Learning Integration:** To adapt NPC strategies based on player history.

9. Conclusion

This project demonstrates how FSMs, a foundational concept from the Theory of Computation, can be effectively applied to model NPC behaviour in video games. The structured approach enables clear, maintainable, and scalable game logic that reacts to real-time player interactions, enhancing the immersive experience. This serves as a practical example of TOC's applicability beyond theoretical domains, extending into modern game development.

Code Implementation

```
import pygame
import math
import random

# Initialize Pygame
pygame.init()
WIDTH, HEIGHT = 800, 600
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Police vs Thief")
clock = pygame.time.Clock()

# Colors
WHITE = (255, 255, 255)
BLUE = (50, 50, 255)    # Thief
RED = (255, 50, 50)     # Police
BLACK = (0, 0, 0)
GRAY = (180, 180, 180)
FONT = pygame.font.SysFont(None, 24)
SMALL_FONT = pygame.font.SysFont(None, 20)

# Thief class
class Thief:
    def __init__(self):
        self.x = WIDTH // 2
        self.y = HEIGHT // 2
        self.speed = 5
        self.radius = 25
        self.health = 10
        self.state = 'Idle'

    def move(self, keys):
        moved = False
```

```
if self.health > 0:
    if keys[pygame.K_LEFT]:
        self.x -= self.speed
        moved = True
    if keys[pygame.K_RIGHT]:
        self.x += self.speed
        moved = True
    if keys[pygame.K_UP]:
        self.y -= self.speed
        moved = True
    if keys[pygame.K_DOWN]:
        self.y += self.speed
        moved = True

if self.health <= 0:
    self.state = "Dead"
elif moved:
    self.state = "Running"
else:
    self.state = "Idle"

def draw(self):
    pygame.draw.circle(screen, BLUE, (int(self.x), int(self.y)), self.radius)
    # Draw name inside circle
    name_text = SMALL_FONT.render("Thief", True, WHITE)
    screen.blit(name_text, (self.x - name_text.get_width() // 2, self.y - name_text.get_height() // 2))
    # Draw state above circle
    state_text = FONT.render(self.state, True, BLACK)
    screen.blit(state_text, (self.x - state_text.get_width() // 2, self.y - self.radius - 25))

# Police class (FSM NPC)
class Police:
    def __init__(self):
        self.x = random.randint(100, 700)
        self.y = random.randint(100, 500)
        self.radius = 30
        self.speed = 2
```



```
self.state = 'Idle'
self.patrol_target = self.random_position()

def random_position(self):
    return random.randint(50, WIDTH - 50), random.randint(50, HEIGHT - 50)

def draw(self, thief_alive):
    color = RED if self.state != 'Dead' else GRAY
    pygame.draw.circle(screen, color, (int(self.x), int(self.y)), self.radius)
    # Draw name inside circle
    name_text = SMALL_FONT.render("Police", True, WHITE)
    screen.blit(name_text, (self.x - name_text.get_width() // 2, self.y - name_text.get_height() // 2))
    # Draw state above circle
    state_text = FONT.render(self.state, True, BLACK)
    screen.blit(state_text, (self.x - state_text.get_width() // 2, self.y - self.radius - 25))

def update(self, thief):
    if thief.health <= 0:
        self.state = 'Dead'
        return

    distance = math.hypot(thief.x - self.x, thief.y - self.y)

    if self.state == 'Idle':
        if random.random() < 0.01:
            self.state = 'Patrolling'

    elif self.state == 'Patrolling':
        self.move_towards(self.patrol_target)
        if self.reached_target(self.patrol_target):
            self.patrol_target = self.random_position()
        if distance < 150:
            self.state = 'Chasing'

    elif self.state == 'Chasing':
        self.move_towards((thief.x, thief.y))
        if distance < 40:
```

```
        self.state = 'Attacking'
    elif distance > 200:
        self.state = 'Patrolling'

    elif self.state == 'Attacking':
        if distance > 40:
            self.state = 'Chasing'
        else:
            thief.health -= 0.05
            if thief.health <= 0:
                thief.health = 0
                thief.state = 'Caught'
                self.state = 'Dead'
            else:
                thief.state = 'Under Attack'

def move_towards(self, target):
    tx, ty = target
    angle = math.atan2(ty - self.y, tx - self.x)
    self.x += self.speed * math.cos(angle)
    self.y += self.speed * math.sin(angle)

def reached_target(self, target):
    return math.hypot(self.x - target[0], self.y - target[1]) < 10

# Game loop
def game():
    thief = Thief()
    police = Police()
    running = True
    end_reason = ""

    while running:
        screen.fill(WHITE)
        clock.tick(60)

        for event in pygame.event.get():
```

```
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    thief.move(keys)
    police.update(thief)

    thief.draw()
    police.draw(thief.health > 0)

    # HUD
    hud_text = FONT.render(f"Thief Health: {int(thief.health)}", True, BLACK)
    screen.blit(hud_text, (10, 10))

    if thief.health <= 0:
        end_reason = "Police caught the Thief!"
        running = False

    pygame.display.flip()

    # End screen
    screen.fill(WHITE)
    message = pygame.font.SysFont(None, 48).render(end_reason, True, RED)
    screen.blit(message, (WIDTH // 2 - message.get_width() // 2, HEIGHT // 2 - 20))
    pygame.display.flip()
    pygame.time.wait(3000)
    pygame.quit()

if __name__ == "__main__":
    game()
```

👮 Police vs Thief

Thief Health: 10

Idle
Police

Idle
Thief

👮 Police vs Thief

Thief Health: 10

Idle
Thief

Patrolling
Police

👮 Police vs Thief

— □ ×

Thief Health: 10



👮 Police vs Thief

— □ ×

Thief Health: 10



👮 Police vs Thief



Thief Health: 4



👮 Police vs Thief



Police caught the Thief!